



Innovation Centre for Education



Yenepoya Institute of Arts Science Commerce and Management

Low Level Design

on

Predictive Analysis of Healthcare Success

Dataset: Heart Disease Prediction

Student Name: Misab

Register Number: 23BCDSBA11

Industry Mentor: Sumith Kumar Shukla

Guided by: Raksha

1. Introduction

1.1 Scope of the Document

- **Data Models & Schema:** Detailed breakdown of the heart.csv dataset — 1,025 patient records across 14 clinical features.
- **Preprocessing Pipeline:** Feature/target separation, train-test splitting strategy, and data validation logic.
- **Algorithm Logic:** In-depth specification of Decision Tree, Random Forest, and Logistic Regression implementations including hyperparameters.
- **Evaluation Design:** Accuracy scoring, confusion matrix computation, and multi-model comparison framework.
- **Visualization Components:** Definitions of all matplotlib/seaborn plots generated during analysis.

1.2 Intended Audience

- **Data Scientists / ML Engineers:** Responsible for implementing, tuning, and extending the prediction pipeline.
- **Quality Assurance (QA) Teams:** To build targeted test cases that validate data integrity, model outputs, and visualization correctness.
- **Academic Reviewers & Mentors:** To assess design decisions, hyperparameter choices, and adherence to ML best practices.

1.3 System Overview

The system ingests a publicly available heart disease dataset (heart.csv), performs exploratory data analysis (EDA), trains multiple classification models, and evaluates their performance to predict healthcare outcomes. The final pipeline achieves 98.54% accuracy using a Random Forest Classifier with 100 estimators. All computation is performed in memory using a Jupyter Notebook environment (Python 3.14, scikit-learn 1.8, pandas 3.0, matplotlib 3.10).

2. System Design

2.1 Application Design

- **Execution Environment:** Jupyter Notebook (.ipynb) running Python 3.14 on a Windows machine.
- **Data Layer:** pandas DataFrame loaded from heart.csv (local filesystem) — no database or ORM required.
- **ML Layer:** scikit-learn estimators (DecisionTreeClassifier, RandomForestClassifier, LogisticRegression) accessed through a unified .fit() / .predict() interface.
- **Visualization Layer:** matplotlib / ConfusionMatrixDisplay for generating inline plots directly within the notebook.

2.2 Process Flow

Step 1 — Data Ingestion:

- `pd.read_csv('heart.csv')` loads 1,025 rows × 14 columns into a DataFrame.
- `df.shape`, `df.head()`, `df.dtypes`, and `df.isnull().sum()` validate structure and confirm zero null values.

Step 2 — Exploratory Data Analysis (EDA):

- Class distribution: 526 positive (heart disease), 499 negative — dataset is balanced.
- Sex distribution: Visualized via pie chart (Male vs. Female split).

Step 3 — Preprocessing:

- `X = df.drop('target', axis=1)` — 13 input features.
- `y = df['target']` — binary output labels.
- `train_test_split(X, y, test_size=0.2, random_state=42)` → 820 training / 205 test samples.

Step 4 — Model Training:

- Primary model: `DecisionTreeClassifier(max_depth=5, random_state=42).fit(X_train, y_train)`.
- Comparison models: `RandomForestClassifier(n_estimators=100)` and `LogisticRegression()`.

Step 5 — Prediction & Evaluation:

- `y_pred = model.predict(X_test)`.
- `accuracy_score(y_test, y_pred)` and `confusion_matrix(y_test, y_pred)` computed.

Step 6 — Visualization:

- `ConfusionMatrixDisplay`, feature importance bar chart, class distribution count plot, sex pie chart.

2.3 Information Flow

- Trigger: Jupyter kernel executes cells sequentially top-to-bottom.
- Data Extraction: pandas reads `heart.csv` from local disk into memory as a structured DataFrame.
- Transformation: Feature/target separation and train-test split produce four numpy arrays.
- Model Computation: scikit-learn estimators train on arrays and return fitted model objects.
- Output Hydration: Predictions are passed to evaluation functions and matplotlib renders inline charts.

2.4 Component Design

- DataLoader Module: Wraps `pd.read_csv()` and basic validation checks (null detection, dtype inspection).
- Preprocessor Module: Handles feature/target separation and sklearn `train_test_split` with fixed random state.

- **ModelTrainer Module:** Instantiates and fits classifiers; encapsulates hyperparameter definitions.
- **Evaluator Module:** Computes accuracy metrics and confusion matrix; runs multi-model comparison loop.
- **Visualizer Module:** Generates all matplotlib figures — confusion matrix heatmap, feature importance, distributions.

2.5 Key Design Considerations

- **Fixed Random State:** `random_state=42` applied to both `train_test_split` and model initializations, guaranteeing reproducible results across all runs.
- **Depth-Limited Decision Tree:** `max_depth=5` prevents overfitting to the training set while maintaining strong generalization.
- **Balanced Dataset:** With 526 vs. 499 class distribution, no SMOTE or class-weight balancing was required.
- **ORM-Free Design:** No database abstraction layer is needed — all data lives in pandas DataFrames for simplicity and speed.
- **In-Memory Processing:** All transformations and predictions occur entirely in RAM; no intermediate files are written.

2.6 API / Function Catalogue

Function / Call	Module	Returns	Description
<code>load_data(path)</code>	DataLoader	DataFrame	Reads heart.csv using <code>pd.read_csv()</code> , returns raw DataFrame (1025 rows × 14 cols)
<code>validate_data(df)</code>	DataLoader	Boolean	Checks for null values and unexpected dtypes; raises <code>ValueError</code> on failure
<code>split_features_target(df)</code>	Preprocessor	X, y	Separates feature columns from target column; returns X (13 features) and y
<code>train_test_split(X,y,0.2)</code>	Preprocessor	X_train, X_test, y_train, y_test	Splits dataset 80/20 with <code>random_state=42</code> for reproducibility
<code>train_model(X_train,y_train)</code>	ModelTrainer	Fitted model	Instantiates <code>DecisionTreeClassifier(max_depth=5, random_state=42)</code> and calls <code>.fit()</code>

Function / Call	Module	Returns	Description
predict(X_test)	ModelTrainer	y_pred array	Calls model.predict() on the test set; returns integer class array
evaluate(y_test,y_pred)	Evaluator	Dict of metrics	Computes accuracy_score and confusion_matrix; returns results dict
compare_models(X,y)	Evaluator	Dict of scores	Trains Decision Tree, Random Forest, Logistic Regression; returns name→accuracy map
plot_confusion_matrix(cm)	Visualizer	Matplotlib Figure	Renders ConfusionMatrixDisplay with 'Blues' colormap
plot_feature_importance(model,X)	Visualizer	Matplotlib Figure	Horizontal bar chart of model.feature_importances_
plot_class_distribution(df)	Visualizer	Matplotlib Figure	Count plot of target classes (526 disease, 499 no-disease)
plot_sex_distribution(df)	Visualizer	Matplotlib Figure	Pie chart of patient sex split

3. Data Design

3.1 Data Model — Feature Schema

Source file: heart.csv | Rows: 1,025 | Columns: 14 | Null values: 0

Feature	Data Type	Value Range	Description
age	int64	Continuous	Patient age in years
sex	int64	Binary (0/1)	0 = Female, 1 = Male
cp	int64	Categorical (0–3)	Chest pain type: 0=typical angina, 1=atypical angina, 2=non-anginal pain, 3=asymptomatic
trestbps	int64	Continuous	Resting blood pressure (mm Hg)

Feature	Data Type	Value Range	Description
chol	int64	Continuous	Serum cholesterol (mg/dl)
fbs	int64	Binary (0/1)	Fasting blood sugar > 120 mg/dl
restecg	int64	Categorical (0–2)	Resting ECG results
thalach	int64	Continuous	Maximum heart rate achieved
exang	int64	Binary (0/1)	Exercise-induced angina
oldpeak	float64	Continuous	ST depression induced by exercise
slope	int64	Categorical (0–2)	Slope of peak exercise ST segment
ca	int64	Categorical (0–4)	Number of major vessels coloured by fluoroscopy
thal	int64	Categorical (1–3)	Thalassemia: 1=normal, 2=fixed defect, 3=reversible defect
target	int64	Binary (0/1)	Output class: 1 = Heart disease, 0 = No heart disease

3.2 Data Access Mechanism

- Primary Access: `pd.read_csv('heart.csv')` — direct filesystem read, returns pandas DataFrame.
- Column Access: `df[col_name]` for single series; `df[list_of_cols]` for multi-column slicing.
- Row Filtering: Boolean indexing e.g. `df[df['target'] == 1]` for class-specific queries.
- No ORM or SQL: All data access is handled natively through pandas indexing and slicing operations.

3.3 Data Retention Policies

- Saving Data: The original heart.csv file is read-only; no modifications are written back to disk.
- In-Memory Lifecycle: All DataFrames, numpy arrays, and fitted model objects exist only for the duration of the Jupyter kernel session.

- Model Persistence: No model serialization (pickle / joblib) is implemented in the current version; models must be retrained on each kernel restart.

3.4 Data Schema Migration

- Current State: Static single-file CSV schema — no migration tooling required.
- Future Extension: If migrated to a relational database, a 'patients' table would map directly to the current feature schema, with 'target' stored as a TINYINT(1) column.

4. Interfaces

- User Interface: Jupyter Notebook cells — users execute cells sequentially and view outputs (text, tables, plots) inline.
- Data Interface: Local filesystem CSV file accessed via pandas; no network or API calls required.
- Model Interface: scikit-learn's unified estimator API (`.fit()`, `.predict()`, `.feature_importances_`) used consistently across all three classifiers.
- Visualization Interface: matplotlib pyplot and ConfusionMatrixDisplay render figures directly in the notebook output area.

5. State and Session Management

- Kernel State: All variables (DataFrames, trained models, predictions) persist in Jupyter kernel memory for the duration of a session.
- Reproducibility: `random_state=42` used consistently across `train_test_split` and all model constructors to ensure identical splits and model weights on every run.
- Session Reset: Restarting the Jupyter kernel clears all state; all cells must be re-executed from top to recreate objects.
- No Authentication: The notebook is a local, single-user tool — no login, session cookies, or access control mechanisms are required.

6. Caching

- DataFrame Caching: Once `heart.csv` is loaded into `df`, it is reused throughout all subsequent cells without re-reading from disk — a natural in-memory cache.
- Model Object Reuse: Trained model objects (`model`, `clf`) are stored as kernel-level variables and reused for prediction and visualization without retraining.
- No Output Caching: Matplotlib plots are regenerated on every cell execution; there is no caching of rendered figures.
- Notebook Cell Outputs: Jupyter caches cell outputs in the `.ipynb` file when saved, allowing results to be viewed without re-executing.

7. Non-Functional Requirements

7.1 Security Aspects

- **Data Privacy:** The heart.csv dataset is anonymized — no patient names, IDs, or directly identifiable information are present.
- **Access Control:** The notebook runs locally on a single machine with no network-exposed endpoints; access is inherently restricted to the local user.
- **No Injection Risk:** All data access is through pandas, which handles CSV parsing safely — there is no SQL or shell injection surface.

7.2 Performance Aspects

- **Training Speed:** Decision Tree and Logistic Regression train in under 1 second on 820 samples. Random Forest (100 estimators) trains in approximately 2–4 seconds.
- **Prediction Speed:** All model predictions on 205 test samples complete in milliseconds — suitable for real-time inference.
- **Memory Footprint:** The entire dataset (1,025 rows $\times$ 14 columns, all numeric) occupies under 1 MB in memory.
- **Scalability Limit:** The current design is optimized for datasets up to ~100K rows; beyond that, chunked reading or a database backend would be required.

7.3 Accuracy & Model Results

Model	Key Hyperparameters	Test Accuracy	Notes
Decision Tree	max_depth=5, random_state=42	84.39%	Simple, interpretable; moderate accuracy; risk of overfitting at deeper depths
Random Forest	n_estimators=100	98.54%	Best performer; ensemble of 100 trees reduces variance; production-recommended
Logistic Regression	Default sklearn params	79.51%	Simplest baseline; assumes linear decision boundary; lowest accuracy

Confusion Matrix — Decision Tree (84.39% accuracy on 205 test samples):

	Predicted: No Disease	Predicted: Disease
Actual: No Disease	TN = 75	FP = 27
Actual: Disease	FN = 5	TP = 98

- True Positive Rate (Sensitivity): $98 / (98+5) = 95.1\%$ — the model correctly identifies 95% of actual heart disease patients.
- Specificity: $75 / (75+27) = 73.5\%$ — relatively higher false positive rate; Random Forest addresses this significantly.
- Recommended Production Model: Random Forest (98.54%) due to its superior accuracy and lower false-positive risk in a clinical context.

8. References

- scikit-learn Documentation: <https://scikit-learn.org/stable/>
- pandas Documentation: <https://pandas.pydata.org/docs/>
- matplotlib Documentation: <https://matplotlib.org/stable/>
- Heart Disease Dataset: UCI Machine Learning Repository — Cleveland Heart Disease Dataset
- Python 3.14 Reference: <https://docs.python.org/3/>